Word count: 990

1 Introduction

For Coursework 3, I developed a drone monitoring dashboard that provides real-time updates about the drone fleet. The aim was to give operators a single interface to view critical drone information without having to rely on backend logging or manual database queries. The original Coursework 2 system calculated the delivery path but provided no insight into the operational state of the drones: if a drone ran out of battery or became stuck mid-flight, operators would have no way of knowing the issue.

The system uses Redis as an in-memory cache, the Spring Boot REST service from Coursework 2 as the primary data source, and a React front-end to fetch and visualise the data. The dashboard displays each drone's battery level, operational status (idle, in-flight, charging, returning), active delivery ID, drone ID, and current coordinates. To enhance usability, I also implemented a delivery order form that calls the existing `calcDeliveryPath` endpoint and updates the drone's status in Redis. This allows the operators to schedule deliveries and monitor the drone status transitions in real time.

2 Innovation and Benefits

The idea came from recognising that a drone could get lost or run out of battery halfway through a flight, and there would be no way for operators to know what had happened and why.

Redis caches drones' status with a 1-hour TTL, and it tracks the complete delivery cycle: `IDLE → IN_FLIGHT → RETURNING → CHARGING → IDLE`. Initially, I had a 5-minute Time To Live (TTL), but this meant that the drones could disappear from the cache during long deliveries. Therefore, I changed it to one hour and made the dashboard update every 5 seconds.

Each delivery path calculated the total moves, so I decided to estimate that the battery would decrease by 0.1% per move. The system pre-computes the battery usage using `totalMoves × 0.1%` and rejects the delivery if it would cause the drone's battery to drop below 10%. During the background simulation, the battery updates as the drone progresses through the path, which allows operators to monitor the drone's state live.

Using the Spring Boot `@Async` annotation, I made `DeliverySimulationService` run a flight simulation that could run in background threads. Every 500ms, the simulation updates the drone's coordinates, decreases the battery level, and writes the new state to Redis. This allows the operators to make new deliveries and switch tabs without interrupting the tracking of the other drones.

3 Implementation

I extended the Coursework 2 service and added several endpoints. For example, I added endpoints to clear and write to the cache as well as to update battery levels. I also integrated the Redis cache into the logic.

Endpoint	Description
POST /syncDronesToCache	Fetches all drones from the ILP REST service and populates Redis with initial states.
POST /calcDeliveryPath	Updates Redis after calculating the path and adjusts the battery percentage.
POST /startSimulation	Initiates the background simulation using `@Async`.
POST /completeDelivery/{droneId}	Transitions the drone from `IN_FLIGHT` to `RETURNING`.
POST /arriveAtBase/{droneId}	Starts recharging the battery and transitions from `IN_FLIGHT` to `CHARGING`.
POST /completeCharging/{droneId}	Sets drone battery to 100% and status to `IDLE`.
GET /drones/active	Returns all cached drones for the dashboard display.
GET /drones/stats	Returns total drone count, in-flight count, and idle count.

Table 1: REST endpoints added for Coursework 3 functionality.

I chose Redis over other options like in-memory Java collections because of the specialised data structures. The system uses three Redis structures: String keys (`drone:status:{id}`) for the DroneStatus objects, a Set (`drones:active`) for membership checking, and a geospatial index (`drone:locations`) for radius-based queries.

I used Coursework 2's A* algorithm, which allows 16 directions of movement and uses ray-casting to check no-fly zones. I also implemented a greedy fallback if A* fails to find a path.

My React front-end has two main components:

- **Drone dashboard**: Displays all drones stored in the cache in a grid layout with updates via HTTP polling every 5 seconds. I chose HTTP polling over WebSockets because drone operations are measured in minutes rather than milliseconds, making this latency acceptable while maintaining efficiency and simplicity.
- **Delivery form**: Accepts an order ID, date, time, requirements, and destination coordinates via an interactive Leaflet map. The form optionally accepts a name and medication for the delivery. Once submitted, it calls `calcDeliveryPath` and receives the GeoJSON path, which triggers `startSimulation` to begin background execution.

4 Limitations

My system does not calculate an estimated time of delivery. Additionally, if Redis crashes without saving, it would cause loss of the most recent data. There are also no automated recovery mechanisms for drones that may become stuck or unresponsive. I also could have added a flight simulation map that shows all deliveries in progress, as coordinates alone may be difficult to visualise.

5 Design Choices

I chose Redis because it provides fast read/write operations, which helps for live monitoring. Unlike querying the REST service directly, which would cause repeated calls and unnecessary load, Redis allows the dashboard to access drone data instantly. Redis also supports horizontal scaling and can handle large volumes of frequently changing data, making it well suited for a growing drone fleet.

The separation between the backend and cache ensures that the Spring Boot REST service maintains control over logic and validation.

I chose React for the front-end because it enables dynamic updates without reloading the page. Since I wanted the drone status, battery level, and location to update as soon as they changed, React was a good option for an efficient user interface. The reusable components also make it easier to add future features.

I decided to display only battery level, status, delivery order ID, location, and drone ID. I felt that these were the metrics operators would use most frequently, and additional fields might introduce unnecessary clutter.

The React dashboard does not store or calculate long-term data itself. Instead, it requests the information from Redis each time, ensuring consistency across users and avoiding stale front-end data.